Favorite fall activity?

Drinks - 41%

clicker.cs.illinois.edu

Q1

~Code~
340

CS 340

Synchronization

Updates

1. MP5 Wallet out
   a. Verify on your VM

2. MP 3.2 graded - file regrades on PL if you see an issue

3. Exam 2 next week
   a. Practice exam - out on PL
   b. Study guide - out on campus wire and website
   c. Tuesday - Review Lecture (practice exam answers)
   d. Thursday - Exam (no class)

4. No HW this week

Synchronization

Today's LGs - Be able to reason about threading, the issues that can appear, and approaches to solving the issues.

Be able to identify simple race conditions and deadlocks.

Be able to add mutexes, conditional variables, and reader/writer locks to a multithreaded program to avoid threading errors.

Agenda

1. Race Conditions
2. Mutexes
3. Reader-writer locks
4. Condition variables
5. Deadlocks

Definitions

Race Condition -

- Threads ← more than 1
- shared resources
- non deterministic behavior

Race Conditions we care about -

- not all race conditions are bad
- where we care the output or order

Does this code have a race condition?

```c
int balance = 0;

void *transaction(void *arg){
    int delta = *((int*)arg);
    balance += delta;
    if(balance < 0){
        //magic, balance can't go negative
        balance = 0;
        printf("ERROR\n");
    }
}

int main() {
    pthread_t threads[3];
    int transactions[3] = {-5, 10, -5};

    for(int i = 0; i < 3; i++){
        pthread_create(&threads[i], NULL, transaction, &transactions[i]);
    }
    for(int i = 0; i < 3; i++){
        pthread_join(threads[i], NULL);
    }
    printf("Your balance is now %i\n", balance);
}
```

clicker.cs.illinois.edu

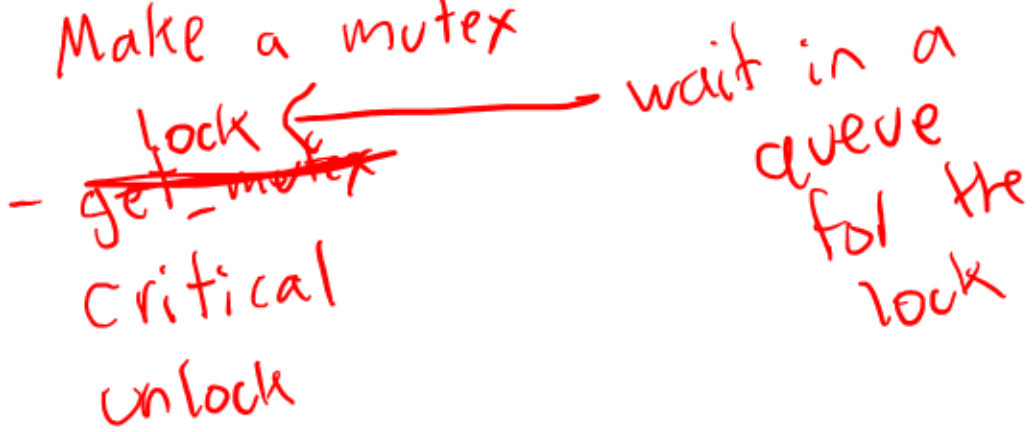
Q2

~Code~
340

Mutex

Definition - mutual exclusion lock, only 1 thread can get the lock

Pattern



Static Mutex Example

```c
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t my_m = PTHREAD_MUTEX_INITIALIZER;

int shared_int = 0;

void *shared_function(void* arg){
    pthread_mutex_lock(&my_m);
    shared_int += 3;
    printf("my val: %i", shared_int);
    shared_int = 0;
    pthread_mutex_unlock(&my_m);
}

//thread creation code and main function
```

Know how many you need at compile time

Static Vs. Dynamic Mutex

Static

```c
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t my_m = PTHREAD_MUTEX_INITIALIZER;
```

Dynamic

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

pthread_mutex_t* my_mutex;

void init_mutex(){
    my_mutex = malloc(sizeof(pthread_mutex_t));
    pthread_mutex_init(my_mutex, NULL);
}

void destroy_mutex(){
    pthread_mutex_destroy(my_mutex);
    free(my_mutex);
}

//code that inits the mutex, uses it, then destroys it
```

If I have an array of 5 items initialized at compile time and I want to protect each of them from race conditions could I use the static initialization method as shown below?

clicker.cs.illinois.edu

Q3

~Code~
340

```
5  pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
6  pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;
7  pthread_mutex_t m3 = PTHREAD_MUTEX_INITIALIZER;
8  pthread_mutex_t m4 = PTHREAD_MUTEX_INITIALIZER;
9  pthread_mutex_t m5 = PTHREAD_MUTEX_INITIALIZER;
```

yes

If I have a linked list data structure and I want to protect each datum from race conditions individually, could I use the static initialization method as shown below?

clicker.cs.illinois.edu

Q4

~Code~ 340

```
5  pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
6  pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;
7  pthread_mutex_t m3 = PTHREAD_MUTEX_INITIALIZER;
8  pthread_mutex_t m4 = PTHREAD_MUTEX_INITIALIZER;
9  pthread_mutex_t m5 = PTHREAD_MUTEX_INITIALIZER;
```

no

Dynamicly

Linked List Example - Protecting the datum

```
5  typedef struct node {
6      struct node* next;
7      pthread_mutex_t* m;
8      int datum;
9  } node;
10
11 typedef struct ll {
12     node* front;
13     node* back;
14 } ll;
```

```
void insert(ll* self, int data){
node new_ = malloc(sizeof(node));
new_->datum = data;
new_->m = malloc(sizeof(pthread_mutex_t));
pthread_mutex_init(new_->m, NULL);
// add to linked list
}
```

```
pop_back()
pthread_mutex_destroy(back->m);
free(back->m);
free(back)
```

Linked List Example - Protecting the datum

```
 5  typedef struct node {
 6      struct node* next;
 7      pthread_mutex_t* m;
 8      int datum;
 9  } node;
10
11  typedef struct ll {
12      node* front;
13      node* back;
14  } ll;
```

```
16  void adjust(ll* self){
17      if(self->front == NULL) return;
18      node* tmp = self->front;
19      pthread_mutex_lock(tmp->m);
20      if(tmp->datum < 0){
21          tmp->datum += 10;
22      }
23      pthread_mutex_unlock(temp->m);
24  }
```

Could there be a race condition error?

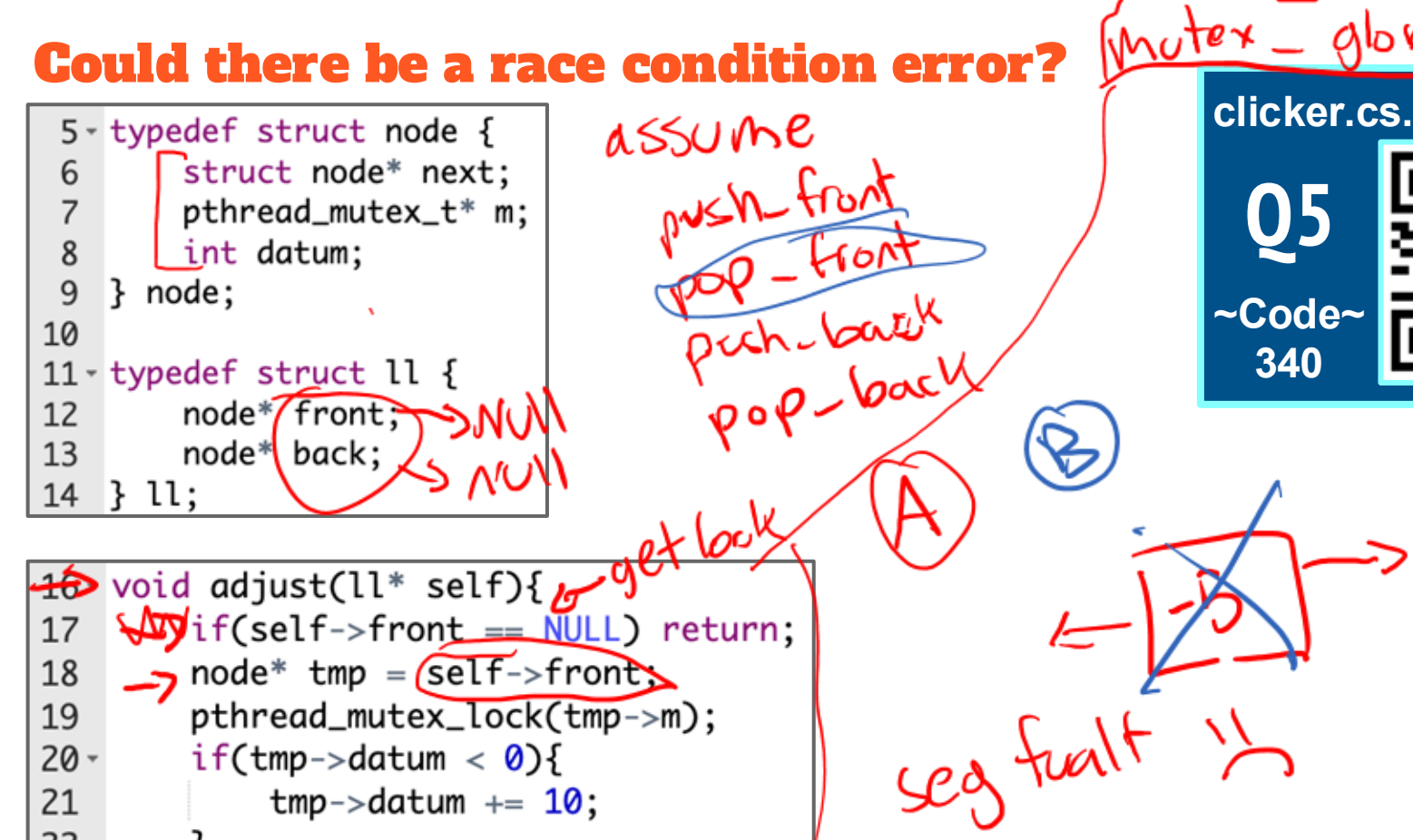
```
5  typedef struct node {
6      struct node* next;
7      pthread_mutex_t* m;
8      int datum;
9  } node;
10
11 typedef struct ll {
12     node* front;
13     node* back;
14 } ll;
```

```
16 void adjust(ll* self){
17     if(self->front == NULL) return;
18     node* tmp = self->front;
19     pthread_mutex_lock(tmp->m);
20     if(tmp->datum < 0){
21         tmp->datum += 10;
22     }
23     pthread_mutex_unlock(temp->m);
24 }
```

clicker.cs.illinois.edu

Q5

~Code~
340



Making a Linked List Thread Safe

Option 1 – 1 mutex for the linked list

slow :(

search the list?

Reader Writer Lock Example

```c
typedef struct node {
  struct node *next;
  //could add mutex
  int data;
} node;

typedef struct {
  node *head;
  pthread_rwlock_t *rwlock;
} my_list;

void my_list_init(my_list *list){
    list->head = NULL;
    list->rwlock = malloc(sizeof(pthread_rwlock_t));
    pthread_rwlock_init(list->rwlock, NULL);
}
```

```c
int find_element(my_list *list, int index){
    pthread_rwlock_rdlock(list->rwlock);
    //read things
    pthread_rwlock_unlock(list->rwlock);
}

void add_element(my_list *list){
    pthread_rwlock_wrlock(list->rwlock);
    //change the structure of the list
    pthread_rwlock_unlock(list->rwlock);
}

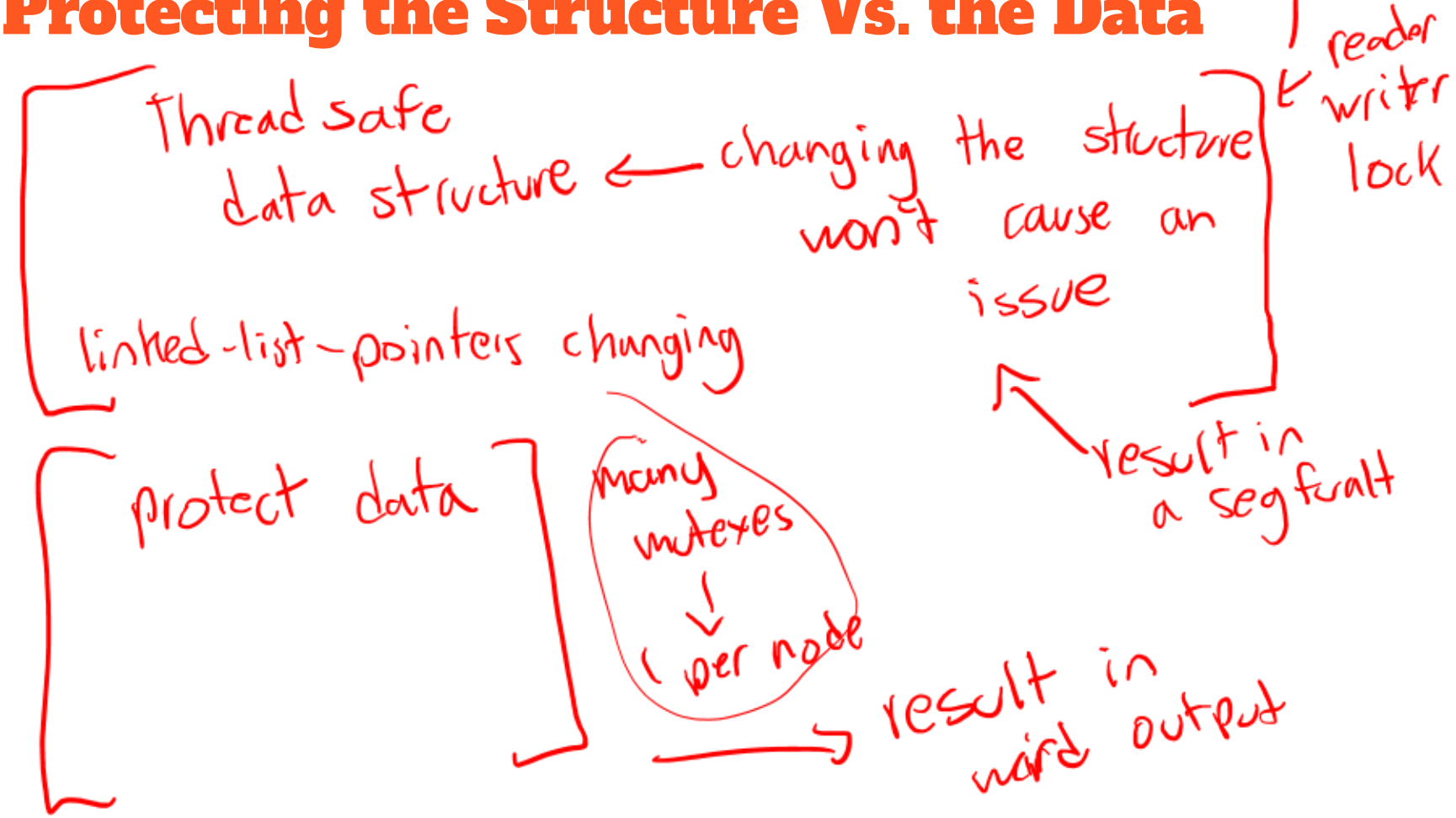
void my_list_destroy(my_list *list){
    //delete all nodes and mutex's
    pthread_rwlock_destroy(list->rwlock);
}
```

free(list->rwlock)

reader – 1 to many at once

writer – only 1 → no readers
no writers

Protecting the Structure Vs. the Data



MP5 - Wallet

We give you a map

1. make map thread safe ← protect structure not elements

   1 reader writer lock

2. protect elements

map

dynamic sorted array

↑

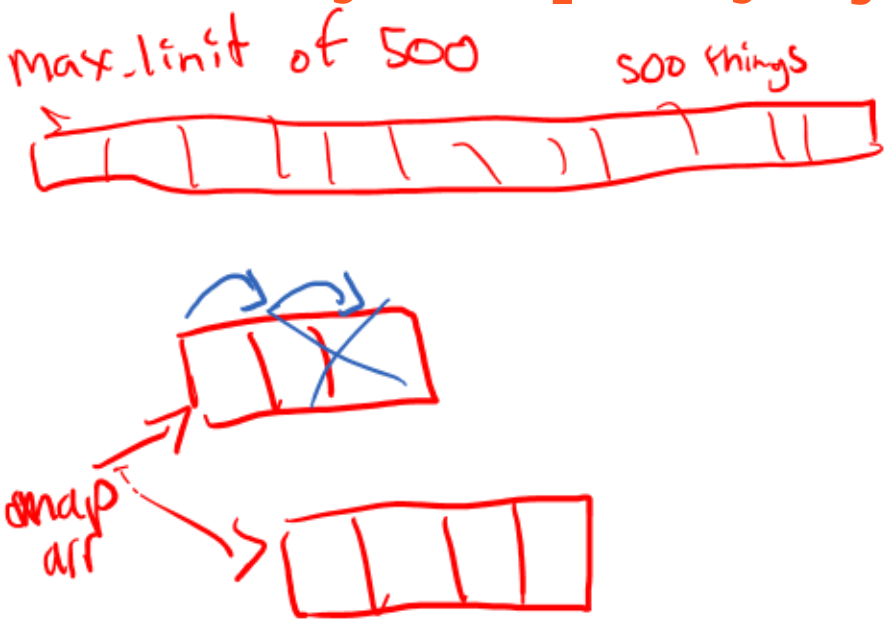
grows if it needs

Do you need synchronization tools to make a map thread safe? Why? Assume the map is implemented with a dynamic array of map entry objects.

clicker.cs.illinois.edu

Q6

~Code~
340

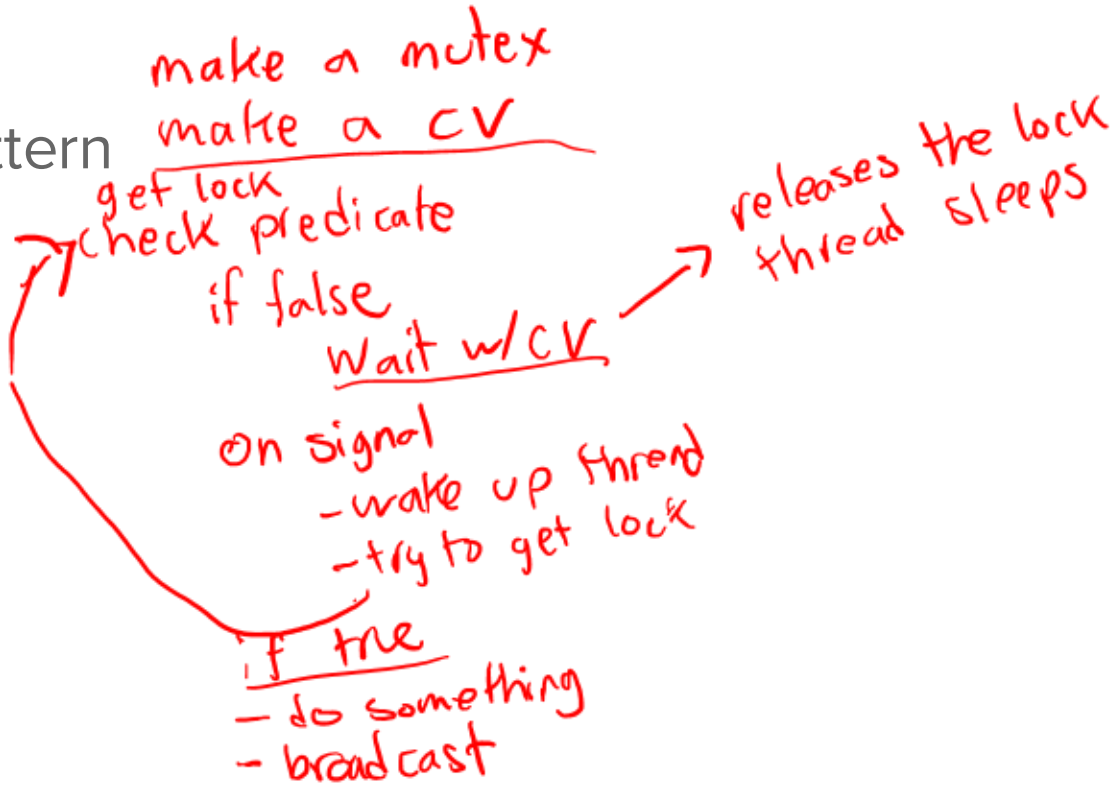


Wallet

Condition Variable

Definition - lets a thread sleep until something is true

Pattern



Static Condition Variable Example

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

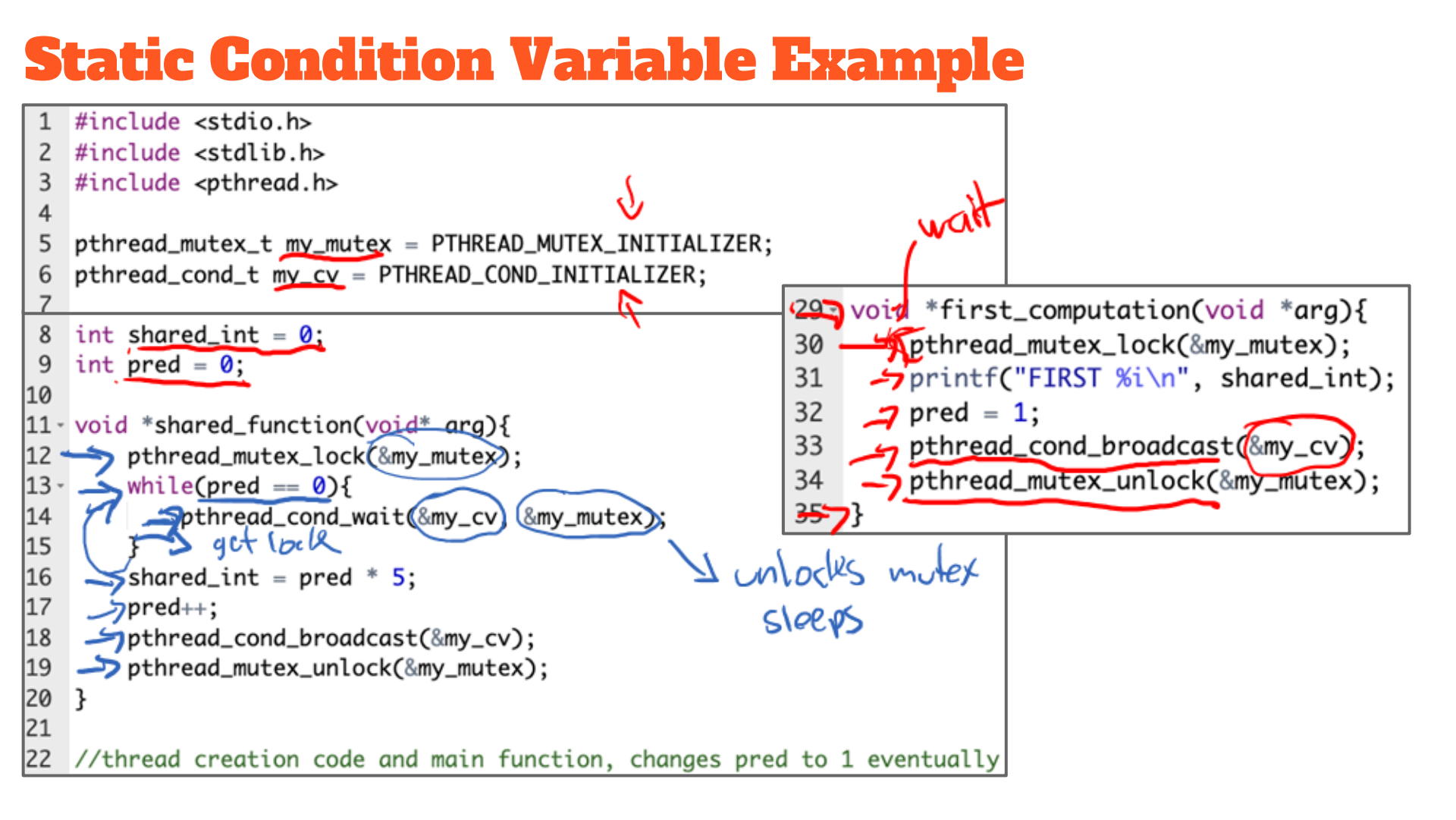
pthread_mutex_t my_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t my_cv = PTHREAD_COND_INITIALIZER;

int shared_int = 0;
int pred = 0;

void *shared_function(void* arg){
    pthread_mutex_lock(&my_mutex);
    while(pred == 0){
        pthread_cond_wait(&my_cv, &my_mutex);
    }
    shared_int = pred * 5;
    pred++;
    pthread_cond_broadcast(&my_cv);
    pthread_mutex_unlock(&my_mutex);
}

//thread creation code and main function, changes pred to 1 eventually
```

```c
void *first_computation(void *arg){
    pthread_mutex_lock(&my_mutex);
    printf("FIRST %i\n", shared_int);
    pred = 1;
    pthread_cond_broadcast(&my_cv);
    pthread_mutex_unlock(&my_mutex);
}
```



Static Vs. Dynamic Condition Variable

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

pthread_mutex_t my_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t my_cv = PTHREAD_COND_INITIALIZER;

```

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

pthread_mutex_t my_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t* my_cv;

int shared_int = 0;
int pred = 0;

void init_cv(){
    my_cv = malloc(sizeof(pthread_cond_t));
    pthread_cond_init(my_cv, NULL);
}

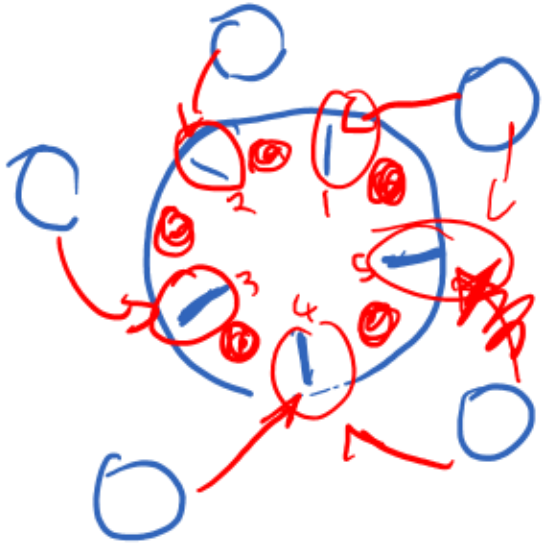
void destroy_cv(){
    pthread_cond_destroy(my_cv);
    free(my_cv);
}

//code inits my_cv, runs code, then destroys cv
```

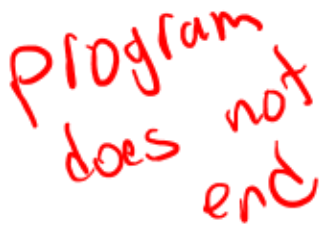
Deadlock



Deadlock - Dining Philosophers



What does deadlock look like in code?



clicker.cs.illinois.edu

Q7

~Code~
340

Deadlock - Example

```
4  int balance_a = 0;
5  int balance_b = 0;
6  pthread_mutex_t mu_a = PTHREAD_MUTEX_INITIALIZER;
7  pthread_mutex_t mu_b = PTHREAD_MUTEX_INITIALIZER;
8
9  void *joint_transaction_a(void *arg){
10     int delta = *((int*)arg);
11     pthread_mutex_lock(&mu_a);
12     pthread_mutex_lock(&mu_b);
13         balance_a = balance_a - balance_b;
14     pthread_mutex_unlock(&mu_a);
15     pthread_mutex_unlock(&mu_b);
16 }
17
18 void *joint_transaction_b(void *arg){
19     int delta = *((int*)arg);
20     pthread_mutex_lock(&mu_b);
21     pthread_mutex_lock(&mu_a);
22         balance_b = balance_b - balance_a;
23     pthread_mutex_unlock(&mu_a);
24     pthread_mutex_unlock(&mu_b);
25 }
```

Deadlock - Example

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

pthread_mutex_t my_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t my_cv = PTHREAD_COND_INITIALIZER;

int shared_int = 0;
int pred = 0;

void *shared_function(void* arg){
    pthread_mutex_lock(&my_mutex);
    while(pred == 0){
        pthread_cond_wait(&my_cv, &my_mutex);
    }
    shared_int = pred * 5;
    pred++;
    pthread_cond_broadcast(&my_cv);
    pthread_mutex_unlock(&my_mutex);
}
```

main spinning up 10 threads

Race Condition - Example

```
5  typedef struct node {
6    struct node *next;
7    //could add mutex
8    int data;
9  } node;
10
11 typedef struct {
12   node *head;
13   pthread_rwlock_t *rwlock;
14 } my_list;
15
16 void my_list_init(my_list *list){
17     list->head = NULL;
18     list->rwlock = malloc(sizeof(pthread_rwlock_t));
19     pthread_rwlock_init(list->rwlock, NULL);
20 }
21
```

```
9  void adjust_element(my_list* list, int target){
10     node *iter = list->head;
11   while(iter){
12     if(iter->data == target){
13       //remove node
14       iter = list->head;
15       continue;
16     }
17     iter = iter->next;
18   }
19 }
```

Deadlock - Example

```c
typedef struct node {
  struct node *next;
  //could add mutex
  int data;
} node;

typedef struct {
  node *head;
  pthread_rwlock_t *rwlock;
} my_list;

void my_list_init(my_list *list){
    list->head = NULL;
    list->rwlock = malloc(sizeof(pthread_rwlock_t));
    pthread_rwlock_init(list->rwlock, NULL);
}
```

```c
void adjust_element(my_list* list, int target){
    pthread_rwlock_rdlock(list->rwlock);
    node *iter = list->head;
    while(iter){
        if(iter->data == target){
            pthread_rwlock_wrlock(list->rwlock);
            //remove node
            pthread_rwlock_unlock(list->rwlock);
            iter = list->head;
            continue;
        }
        iter = iter->next;
    }
    pthread_rwlock_unlock(list->rwlock);
}
```

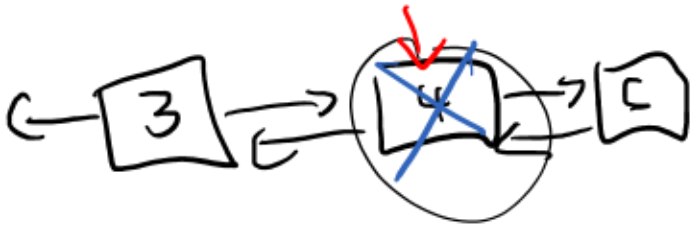
Race Condition - Example

```c
typedef struct node {
  struct node *next;
  //could add mutex
  int data;
} node;

typedef struct {
  node *head;
  pthread_rwlock_t *rwlock;
} my_list;

void my_list_init(my_list *list){
    list->head = NULL;
    list->rwlock = malloc(sizeof(pthread_rwlock_t));
    pthread_rwlock_init(list->rwlock, NULL);
}
```

```c
void adjust_element(my_list* list, int target){
  pthread_rwlock_rdlock(list->rwlock);
  node *iter = list->head;
  while(iter){
    if(iter->data == target){
      pthread_rwlock_unlock(list->rwlock);
      pthread_rwlock_wrlock(list->rwlock);
      //remove node
      pthread_rwlock_unlock(list->rwlock);
      pthread_rwlock_rdlock(list->rwlock);
      iter = list->head;
      continue;
    }
    iter = iter->next;
  }
  pthread_rwlock_unlock(list->rwlock);
}
```



Bad Fix

```c
typedef struct node {
  struct node *next;
  //could add mutex
  int data;
} node;

typedef struct {
  node *head;
  pthread_rwlock_t *rwlock;
} my_list;

void my_list_init(my_list *list){
    list->head = NULL;
    list->rwlock = malloc(sizeof(pthread_rwlock_t));
    pthread_rwlock_init(list->rwlock, NULL);
}
```

```c
void adjust_element(my_list* list, int target){
    pthread_rwlock_wrlock(list->rwlock);
    node *iter = list->head;
    while(iter){
        if(iter->data == target){
            //remove node
            iter = list->head;
            continue;
        }
        iter = iter->next;
    }
    pthread_rwlock_unlock(list->rwlock);
}
```

slow

Good Fix

```
5  typedef struct node {
6    struct node *next;
7    //could add mutex
8    int data;
9  } node;
10
11 typedef struct {
12   node *head;
13   pthread_rwlock_t *rwlock;
14 } my_list;
15
16 void my_list_init(my_list *list){
17     list->head = NULL;
18     list->rwlock = malloc(sizeof(pthread_rwlock_t));
19     pthread_rwlock_init(list->rwlock, NULL);
20 }
21
```

```
9  void adjust_element(my_list* list, int target){
10     pthread_rwlock_rdlock(list->rwlock);
11     node *iter = list->head;
12     while(iter){
13       if(iter->data == target){
14         pthread_rwlock_unlock(list->rwlock);
15         pthread_rwlock_wrlock(list->rwlock);
16         node *iter_2 = list->head;
17         while(iter_2){
18           if(iter->data == target){
19             //remove node
20             break;
21           }
22         }
23         pthread_rwlock_unlock(list->rwlock);
24         pthread_rwlock_rdlock(list->rwlock);
25         iter = list->head;
26         continue;
27       }
28       iter = iter->next;
29     }
30
31     pthread_rwlock_unlock(list->rwlock);
32 }
```

Summary

Errors -

Synchronization tools -